



Análise Didática - LojaExpress Pro

Entendendo os conceitos fundamentais de programação



Variáveis



Constantes



Entradas



Saídas



Processamento



Condicionais



Repetições



Funções



Visão Geral para Iniciantes

Este é um **sistema completo de loja virtual** com dois perfis: **Cliente** (compra) e **Admin** (gerencia). Vamos destrinchar os conceitos fundamentais de programação usados neste código.



Dica: Cada seção abaixo apresenta um conceito com exemplos reais do código-fonte!

1 Variáveis

Variáveis são "**caixas**" onde guardamos valores que podem mudar durante a execução.



Exemplos no código:

```
// Variável simples com número
let contador = 0;

// Variável com texto (string)
let nomeCliente = "Ana Beatriz";

// Variável com número decimal (preço)
let preco = 32.00;

// Variável booleana (verdadeiro/falso)
let adminLogado = false;

// Variável que guarda uma lista (array)
let carrinho = [];

// Variável que guarda um objeto (vários dados juntos)
let clienteLogado = {
  id: 1,
  nome: "Ana",
```

🎯 Variável principal do código — state:

```
let state = {  
  visao: 'cliente',           // Qual tela está ativa  
  clienteLogado: null,        // Cliente logado ou null  
  adminLogado: false,         // Admin logado?  
  cart: [],                   // Itens do carrinho  
  cupomAplicado: null,        // Cupom ativo  
  produtoDetalheAtivo: null,  // Produto sendo visto  
  qtdDetalhe: 1               // Quantidade selecionada  
};
```

💡 **Conceito:** Esta variável state guarda **tudo o estado** do aplicativo. É como um "cérebro" que sabe o que está acontecendo.

2 Constantes

Constantes são "caixas" que **NÃO PODEM SER ALTERADAS** depois de criadas. Usamos `const`.

```
// Constante com lista de produtos iniciais  
const SEED_PRODUTOS = [  
  { id: 101, nome: "Hambúrguer", preco: 32.00, estoque: 20 },  
  { id: 102, nome: "Pizza", preco: 58.00, estoque: 15 }  
];  
  
// Constante com configurações  
const SEED_CONFIG = {  
  adminEmail: "admin@loja.com",  
  adminSenha: "admin123"  
};
```

💡 **Por que usar constantes?** Porque estes valores **nunca devem mudar** durante a execução. Se tentar mudar, o JavaScript dá erro — isso evita bugs!

3 Entradas (Input)

Entrada é quando o programa **recebe dados do usuário**.

a) Digitação em campos de texto:

```
// Pega o que o usuário digitou no campo de busca
const textoBusca = document.getElementById("input-busca").value;

// Pega o e-mail digitado no login
const email = document.getElementById("cli-log-email").value.trim();

// Pega o valor digitado no cupom
const codigo = document.getElementById("input-cupom").value.trim().toUpperCase();
```

b) Cliques em botões:

```
<!-- Quando o usuário clica, a função é chamada -->
<button onclick="adicionarDetalheAoCarrinho()">Adicionar ao Carrinho</button>
<button onclick="abrirDetalhe(101)">Ver Detalhes</button>
```

c) Seleção em dropdowns (select):

```
// Pega o status selecionado no dropdown do admin
const novoStatus = document.getElementById("adm-status").value;
```

4 Saídas (Output)

Saída é quando o programa **mostra informações** para o usuário.

a) Exibir na tela (HTML):

```
// Mostra o nome do produto na tela
document.getElementById("detalhe-nome").innerText = p.nome;

// Cria a lista de produtos no grid
grid.innerHTML = produtos.map(p => `
    <div class="product-card">${p.nome}</div>
`).join('');
```

b) Mostrar mensagem temporária (Toast):

```
function mostrarToast(mensagem) {
    const t = document.getElementById("app-toast");
    t.innerText = mensagem; // Coloca o texto
    t.style.display = "block"; // Mostra
    setTimeout(() => t.style.display = "none", 2500); // Esconde após 2.5s
}
```

```
// Uso:  
mostrarToast("🍔 Produto adicionado!");
```

c) Alertas simples:

```
alert("E-mail ou senha incorretos.");
```

5 Processamento

Processamento é o **cálculo ou transformação** de dados.

Exemplo 1 — Calcular total do carrinho:

```
function calcularTotaisCarrinho() {  
  // Soma todos os subtotais  
  const subtotal = state.cart.reduce((acc, item) =>  
    acc + (item.preco * item.qtd), 0  
  );  
  
  // Calcula desconto  
  let desconto = 0;  
  if (state.cupomAplicado) {  
    if (state.cupomAplicado.tipo === 'pct')  
      desconto = subtotal * (state.cupomAplicado.valor / 100);  
    if (state.cupomAplicado.tipo === 'reais')  
      desconto = state.cupomAplicado.valor;  
  }  
  
  // Calcula total final  
  const total = Math.max(0, subtotal - desconto + 10);  
  
  return { subtotal, desconto, total };  
}
```

Exemplo 2 — Dar baixa no estoque:

```
// Diminui a quantidade do estoque após uma venda  
state.cart.forEach(item => {  
  const produto = prods.find(x => x.id === item.id);  
  if (produto) {  
    produto.estoque = Math.max(0, produto.estoque - item.qtd);  
  }  
});
```

6 Estruturas Condicionais

Permitem que o programa **tome decisões** (escolha caminhos).

a) if / else if / else — Múltiplas escolhas:

```
if (pedido.status === 'Entregue') {  
  // Cliente PODE avaliar  
  liberarAvaliacao();  
} else if (pedido.status === 'Em Transporte') {  
  // Aguardando entrega  
  mostrarMensagem("A caminho!");  
} else {  
  // Outros status  
  mostrarMensagem("Processando...");  
}
```

b) if simples — Uma verificação:

```
// Verifica se o cliente está logado  
if (!state.clienteLogado) {  
  abrirModalAuthCliente();  
  alert("Identifique-se para fechar o pedido!");  
  return;  
}
```

c) Operador ternário — Decisão curta:

```
// Se estoque > 0, mostra verde; senão, vermelho  
const cor = (p.estoque > 0) ? 'green' : 'red';
```

7 Estruturas de Repetição

Servem para **repetir tarefas** sem escrever várias vezes.

a) forEach — Para cada item da lista:

```
// Para CADA produto do carrinho, dá baixa no estoque  
state.cart.forEach(item => {  
  const produto = prods.find(x => x.id === item.id);  
  produto.estoque -= item.qtd;  
});
```

b) map — Transformar cada item:

```
// Transforma CADA produto em HTML
grid.innerHTML = produtos.map(p => `
  <div class="product-card">
    <h3>${p.nome}</h3>
    <p>R$ ${p.preco}</p>
  </div>
`).join('');
```

c) reduce — Reduzir para um valor único:

```
// Soma todos os preços em um único total
const subtotal = state.cart.reduce(
  (acumulador, item) => acumulador + (item.preco * item.qtd),
  0
);
```

d) for tradicional — Repetir N vezes:

```
for (let i = 0; i < 10; i++) {
  console.log("Rodada número " + i);
}
```

8 Funções

Funções são blocos de código que executam uma tarefa específica. Servem para **não repetir código**.

a) Função simples sem parâmetros:

```
function atualizarBadgeCarrinho() {
  const totalItens = state.cart.reduce((a, b) => a + b.qtd, 0);
  document.getElementById("badge-cart-count").innerText = totalItens;
}
```

b) Função com parâmetros (recebe dados):

```
// Recebe um ID e abre o produto
function abrirDetalhe(id) {
  const produto = getProdutos().find(i => i.id === id);
}
```

```
// ... monta a tela com o produto
}
```

c) Função que retorna valor:

```
function getProdutos() {
  return JSON.parse(localStorage.getItem("db_produtos")) || [];
}
```

d) Arrow function (função anônima):

```
// Sem nome, usada só uma vez
state.cart.forEach(item => {
  console.log(item.nome);
});
```

🎯 Resumo Visual - Mapa do Código

Conceito	Onde encontrar	Exemplo
Variáveis	Em todo o código	let state = {...}, let total = 0
Constantes	Início do script	const SEED_PRODUTOS = [...]
Entrada	Formulários e eventos	document.getElementById("email").value
Saída	innerText, innerHTML, alert	mostrarToast("Sucesso!")
Processamento	Funções de cálculo	calcularTotaisCarrinho()
Condicional	if/else, ternário, switch	if (estoque > 0)
Repetição	forEach, map, reduce, for	produtos.map(p => ...)
Funções	Nome + parâmetros + return	function abrirDetalhe(id) { ... }

💡 Dica Final para Iniciantes

Programar é como montar um LEGO:

- **Variáveis** são as peças (guardam dados)
- **Funções** são os blocos prontos (fazem tarefas)
- **Condicionais** são escolhas (qual peça usar?)
- **Repetições** são ações repetidas (montar várias peças iguais)
- **Entradas** são o que você recebe (instruções)
- **Saídas** são o resultado (o brinquedo montado!)

🚀 Comece pequeno: **faça um programa que pede o nome do usuário e exibe "Olá, [nome]!"**. Depois vá aumentando a complexidade!

Quiz - Fixação de Conceitos

Responda as **10 questões** sobre os conceitos apresentados.

Ao final, clique em "**Verificar Respostas**" para corrigir.

1 Qual palavra-chave usamos para declarar uma **variável** que pode mudar de valor?

☐ const

☒ let

☐ function

☐ return

2 Qual palavra-chave usamos para declarar uma **constante** que **NÃO** pode ser alterada?

☐ let

☐ var

☒ const

☐ static

3 O que é uma **ENTRADA**(input) em um programa?

☐ Um cálculo que o programa faz

☒ Um dado que o usuário fornece ao programa

☐ Uma mensagem que o programa exibe

☐ Uma função matemática

4 No código `document.getElementById("input-busca").value`, o que isso representa?

☐ Uma saída de dados

☒ Uma entrada de dados do usuário

☐ Uma estrutura de repetição

☐ Uma constante do sistema

5 Qual estrutura condicional usamos quando temos **múltiplas escolhas** a testar?

☐ for

☒ if / else if / else

☐ while

☐ forEach

6 Qual método de arrays **soma todos os valores** em um único resultado?

☐ forEach

☐ map

☐ filter

☒ reduce

7 Qual método de array **transforma cada item** em um novo valor?

☒ map

☐ reduce

☐ find

☐ every

8 O que é uma **SAÍDA**(output) em um programa?

☐ Um dado que o usuário digita

☒ Uma informação que o programa exibe para o usuário

☐ Uma variável que guarda números

☐ Um arquivo do sistema

9 Qual é a principal **vantagem de usar FUNÇÕES** no código?

☐ Deixar o código mais lento

☒ Evitar repetir o mesmo código várias vezes

☐ Aumentar o tamanho do arquivo

☐ Impedir a entrada de dados

10 Sobre a variável **state** do código, qual afirmação está correta?

☐ Ela guarda apenas o nome do cliente

☐ Ela guarda apenas os produtos da loja

☒ Ela guarda o estado geral do aplicativo (carrinho, usuário, cupom, etc)

☐ Ela é uma constante que nunca muda

 Verificar Respostas Limpar respostas

Resultado do Quiz

10 / 10

Feedback das respostas:

✓ Q1 - Correta

let declara variáveis que podem ser alteradas. const é para constantes.

✓ Q2 - Correta

const cria constantes — valores que não podem ser reatribuídos.

✓ Q3 - Correta

Entrada é quando o programa recebe dados do usuário (digitação, cliques, seleções).

✓ Q4 - Correta

document.getElementById().value captura o que o usuário digitou — é uma ENTRADA.

✓ Q5 - Correta

if/else if/else permite testar múltiplas condições em sequência.

✓ Q6 - Correta

reduce() percorre o array e acumula os valores em um único resultado.

✓ Q7 - Correta

map() percorre o array e transforma cada item em um novo valor.

✓ Q8 - Correta

Saída é o programa exibindo informações (innerText, innerHTML, alert, toast).

✓ Q9 - Correta

Funções evitam repetir código — você escreve uma vez e usa várias vezes.

✓ Q10 - Correta

state é um objeto que guarda o estado geral: carrinho, cliente logado, cupom, etc.



PARABÉNS! Você acertou todas! Excelente domínio dos conceitos!



Imprimir (PDF)



Recomeçar

